

Algebra & *linear algebra.*

From one number to a matrix.

Understand the calculations that connect AI.

🕒 25-30 min core reading · Start with arithmetic · 12 short sections

Linear algebra gives you a compact language for the calculations inside AI. Start with a few numbers you can check by hand, then recognize the same operations in larger models.

WHAT YOU WILL BE ABLE TO DO

Read a prediction equation, check array shapes, calculate a dot product, and explain a matrix transformation. Distinguish length from direction and recognize when a calculation loses information.

Reading path: Work through one small prediction, explore a two-dimensional transformation, then run the Python workbook. Basic arithmetic is enough to begin. Optional notes introduce broadcasting, linear systems, and decompositions without requiring them for the core exercises.

1. Read an equation as a calculation

Algebra uses symbols to describe relationships. A **variable** stands for a value; a **function** maps an input to an output. In the familiar toy predictor:

$$f(x) = wx + b$$

x is the input, w is its weight, and b is the **bias** or intercept. With $w = 2$ and $b = 1$, entering $x = 3$ gives $2 \times 3 + 1 = 7$. Multiplication happens before addition. The function is not learned merely because we have written it as an equation.

If you know an output y and w is nonzero, subtract b from both sides and divide by w :

$$y = wx + b \implies x = \frac{y - b}{w}$$

For $y = 7$, this gives $x = (7 - 1) / 2 = 3$. When $w = 0$, every input returns b , so the output cannot identify a unique input.

Strictly, multiplication by w is **linear**; adding a nonzero b makes the map **affine**. A linear map preserves addition and scaling and sends zero to zero. The familiar “linear regression” model usually includes an intercept despite that name.

▼ Math refresher: powers, roots, logarithms, and subscripts

A power repeats multiplication: $2^3 = 2 \times 2 \times 2 = 8$. The nonnegative square root reverses squaring a nonnegative number: $\sqrt{9} = 3$, while $(-3)^2$ is also 9. Parentheses matter. In Python, use `**` for a power; `^` is not exponentiation.

A logarithm asks for the exponent: $\log_2(8) = 3$ because $2^3 = 8$. Real logarithms require a positive argument and a positive base other than 1. The natural logarithm uses the base e , approximately 2.718. We will use logs in later probability and loss calculations.

Subscripts identify entries: x_1 means the first entry, not x multiplied by 1. A hat, as in $\hat{y}$, means a prediction.

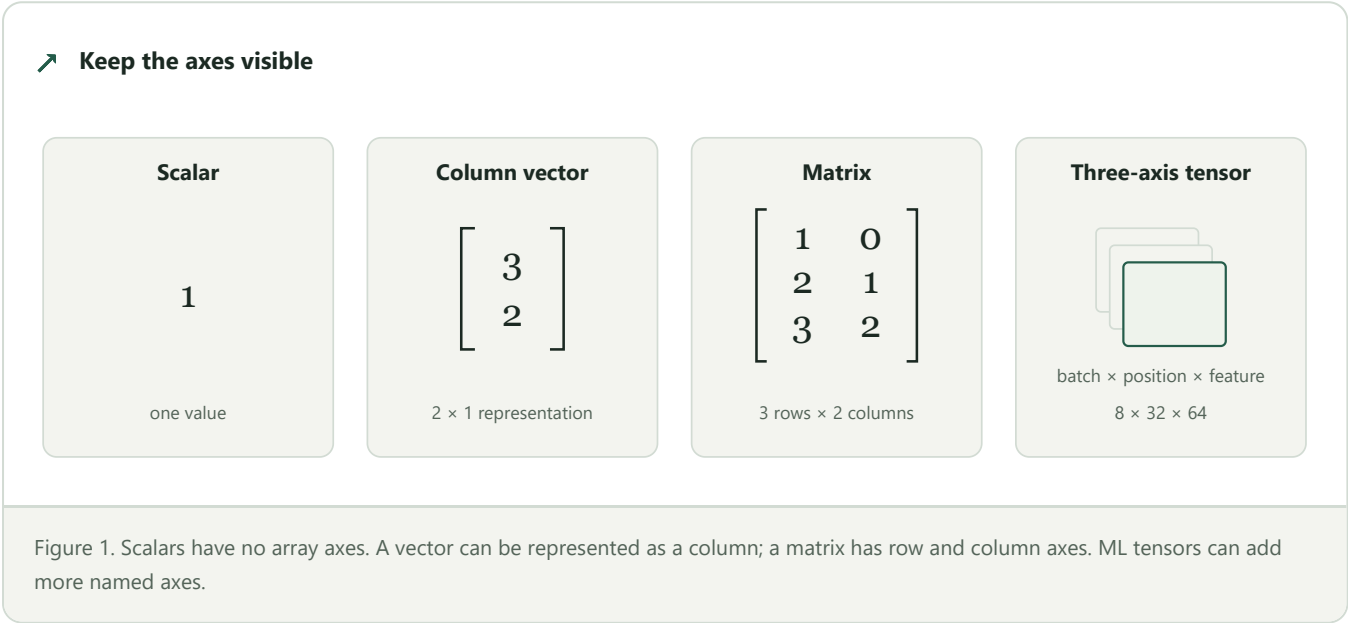
Mathematical indices often start at 1; Python list indices start at 0.

2. Know what object each symbol represents

Linear algebra studies vectors, matrices, and linear maps. Keep the type of numerical object separate from its values:

Object	Meaning here	Example shape
Scalar	One number, such as a bias	No array axes
Vector	An ordered collection of coordinates	2 entries
Matrix	A rectangular array, indexed by row and column	3×2 : three rows, two columns
Tensor in an ML library	An array that can have multiple axes	$8 \times 32 \times 64$: three axes

The **shape** records the size of each axis. A vector with two coordinates belongs to a two-dimensional space; that does not mean a library must store it as a two-axis array. A column representation has shape 2×1 , whereas a NumPy one-dimensional array has shape `(2,)`.



For this chapter’s synthetic runtime example, let $\mathbf{x} = [3, 2]$ mean **[file size in MB, number of queued jobs]**. Feature order is part of the contract. Swapping the entries changes their meaning even when the shape still matches. This extends Chapter 5’s example with a second feature; it is not real performance data.

A batch of 8 sequences, each with 32 positions and 64 numerical features, can be stored as an $8 \times 32 \times 64$ tensor. Name the axes before doing arithmetic. The [D2L linear algebra introduction](#) gives more examples of these objects.

3. Add, scale, and measure a vector

For geometry, use dimensionless coordinates $\mathbf{u} = [3, 4]$ and $\mathbf{v} = [1, -1]$, separate from the runtime features. Add corresponding entries: $\mathbf{u} + \mathbf{v} = [4, 3]$. Multiply every entry to scale: $2\mathbf{u} = [6, 8]$. A negative scalar reverses direction; multiplying by zero produces the zero vector.

A **linear combination** scales vectors and adds them. For example, $2\mathbf{u} + \mathbf{v} = [7, 7]$. Geometrically, place arrows head to tail to add them.

➤ The same coordinates explain length and addition

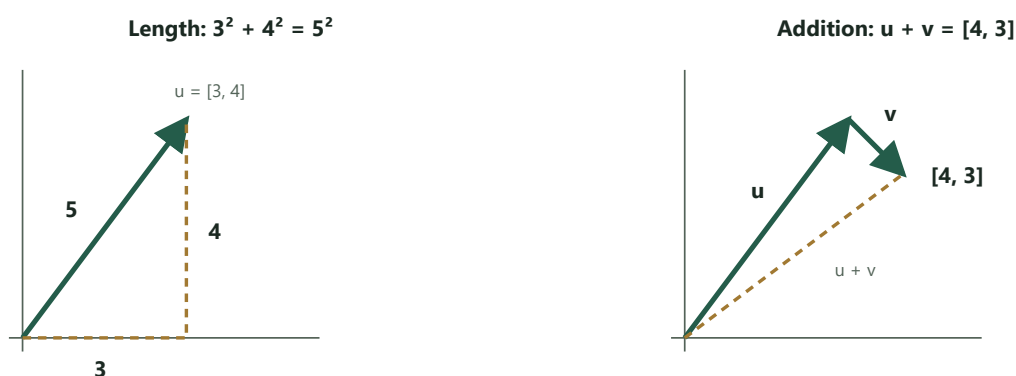


Figure 2. Left: the right triangle gives the length of $u = [3, 4]$ as 5. Right: place $v = [1, -1]$ at the tip of u to reach $u + v = [4, 3]$.

The **Euclidean norm**, written with double bars, measures length:

$$\|u\|_2 = \sqrt{3^2 + 4^2} = 5$$

The **Euclidean distance** between two vectors is the length of their difference. Here $u - v = [2, 5]$, so their distance is $\sqrt{29}$, approximately 5.385. Length is a distance from zero; distance between two vectors need not equal either vector's length.

For nonzero u , dividing by its norm creates a **unit vector**: $u / 5 = [0.6, 0.8]$, of length 1. This preserves direction while removing magnitude. The zero vector cannot be normalized this way.

▼ Go deeper: norms and feature scaling

The L1 norm adds absolute values: for $[3, -4]$, it is 7. The L2 norm is $\sqrt{9 + 16} = 5$. The infinity norm takes the largest absolute entry: 4. These measure size in different ways and appear in distance measures and regularization.

Do not blindly measure distance between raw features with different units. A coordinate measured in bytes can dominate one measured in seconds. Feature scaling uses a stated convention to make comparisons appropriate; fit any learned scaling statistics on training data, then reuse them for validation and test data.

Unit-length normalization acts on a vector's magnitude. Standardizing a feature using its mean and standard deviation is a different operation, introduced with statistics next chapter.

4. Use a dot product to combine evidence

The **dot product** multiplies corresponding coordinates and sums the results. It produces one scalar, not another vector. With the two runtime features $x = [3, 2]$, weights $w = [2, 0.5]$, and bias $b = 1$:

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b = 2 \times 3 + 0.5 \times 2 + 1 = 8 \text{ seconds}$$

Read the two contributions separately: 6 seconds associated with file size, 1 second with queue count, then the 1-second intercept. The weights carry the matching units: seconds per MB and seconds per queued job. The numbers are chosen for arithmetic, not estimated from observations.

The superscript T means **transpose**. It turns the column-vector representation of $\mathbf{w}$ into a row, so a 1×2 row can multiply a 2×1 column. We also write the same operation as $\mathbf{w} \cdot \mathbf{x}$.

For d coordinates, the compact notation is:

$$\mathbf{w} \cdot \mathbf{x} = \sum_{j=1}^d w_j x_j$$

The large sigma means “add these terms for j from 1 through d .” For $d = 2$, it expands to $w_1 x_1 + w_2 x_2$. Matching lengths are necessary, but matching feature order and meaning are just as important.

An **elementwise product** keeps the individual products: $[3, 2]$ multiplied entry by entry by $[2, 0.5]$ gives $[6, 1]$. The dot product adds those values to get 7; the bias is a separate addition.

5. Separate direction from magnitude

For nonzero real vectors, the dot product also relates length and angle:

$$\cos(\theta) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}$$

Here θ is the angle between the vectors. This ratio is **cosine similarity**. Consider dimensionless vectors:

Pair	Dot product	Cosine similarity	Interpretation
$[3, 4]$ and $[6, 8]$	50	1	Same direction; second vector is twice as long
$[3, 4]$ and $[-4, 3]$	0	0	Perpendicular, or orthogonal
$[3, 4]$ and $[-3, -4]$	-25	-1	Opposite directions

For the first row, the lengths are 5 and 10, so $50 / (5 \times 10) = 1$. Its Euclidean distance is 5 even though its cosine similarity is 1. The two measures answer different questions.

Cosine similarity ranges from -1 to 1 ; it is not a probability. With a zero vector, its denominator is zero and the mathematical value is undefined. A library’s fallback value must not be mistaken for a meaningful angle.

Later, embeddings will represent documents as vectors. Similarity is useful only relative to the representation and task: geometrically close vectors do not, by themselves, establish factual agreement or answer correctness.

6. Turn a batch into a matrix calculation

Put one example in each row of a **design matrix** X . Here its two columns mean size in MB and queued jobs:

$$X = \begin{bmatrix} 1 & 0 \\ 2 & 1 \\ 3 & 2 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} 2 \\ 0.5 \end{bmatrix}, \quad b = 1$$

X has shape 3×2 ; $\mathbf{w}$ has shape 2×1 . Matrix-vector multiplication takes a dot product with each row:

$$X\mathbf{w} = \begin{bmatrix} 2 \\ 4.5 \\ 7 \end{bmatrix}, \quad \hat{\mathbf{y}} = X\mathbf{w} + b\mathbf{1} = \begin{bmatrix} 3 \\ 5.5 \\ 8 \end{bmatrix}$$

The vector $\mathbf{1}$ contains three ones, so b is added to every prediction. The general multiplication rule is $(\mathbf{m} \times \mathbf{n})(\mathbf{n} \times \mathbf{p}) \rightarrow (\mathbf{m} \times \mathbf{p})$: inner dimensions match; outer dimensions describe the result. Each output entry is one row-column dot product.

➤ One row-column calculation for each result

$$\begin{array}{c} X \cdot 3 \times 2 \\ \left[\begin{array}{cc} 1 & 0 \\ 2 & 1 \\ 3 & 2 \end{array} \right] \end{array} \times \begin{array}{c} \mathbf{w} \cdot 2 \times 1 \\ \left[\begin{array}{c} 2 \\ 0.5 \end{array} \right] \end{array} = \begin{array}{c} X\mathbf{w} \cdot 3 \times 1 \\ \left[\begin{array}{c} 2 \\ 4.5 \\ 7 \end{array} \right] \end{array}$$

Highlighted row: $3 \times 2 + 2 \times 0.5 = 7$

Figure 3. Three rows, two input features, one output per row. The matching inner size is 2. These outputs are before the separate $+1$ bias.

The transpose X^T has shape 2×3 : rows become columns, without changing the entries' values. Transpose is not inversion. Reversing multiplication is generally different: AB may differ from BA , or one order may not be defined.

In NumPy, `@` performs matrix multiplication and `*` performs elementwise multiplication. Plain Python lists do not implement matrix multiplication. See [NumPy's matmul rules](#).

▼ Go deeper: broadcasting and shape bugs

Broadcasting lets an array operation reuse values across compatible axes. A scalar bias can be added across a prediction array without manually copying it. NumPy compares shapes from the last axis backwards: dimensions must be equal or one of them must be 1; missing leading axes act like size 1.

Beware: an array with shape `(3, 1)` minus one with shape `(3,)` produces a `(3, 3)` array, not three pairwise errors. Explicitly align the shapes of predictions and targets before computing a loss.

A one-dimensional NumPy array with shape `(2,)` does not become a column when transposed; it stays one-dimensional. Use an explicit 2×1 representation when that distinction matters. See [the broadcasting guide](#).

7. See a matrix change a vector

A matrix can represent a **linear transformation**. Its columns show where the coordinate basis vectors go. In two dimensions, those basis vectors are $[1, 0]$ and $[0, 1]$. If $u = [2, 1]$, then Au is twice A 's first column plus once its second column.

PREDICT THE CHANGE

[Live matrix calculation](#)

One vector. Five transformations.

Choose a matrix and change the input coordinates. The page calculates Au directly; no model is trained. These coordinates have no physical units.

Worked input $u = [2, 1]$. In every case, take a dot product with each row of A :

Transformation	Rows of A	Output Au
Identity	$[1, 0]; [0, 1]$	$[2, 1]$
Stretch horizontally	$[2, 0]; [0, 1]$	$[4, 1]$
Rotate 90° counterclockwise	$[0, -1]; [1, 0]$	$[-1, 2]$
Shear horizontally	$[1, 1]; [0, 1]$	$[3, 1]$
Project onto the horizontal axis	$[1, 0]; [0, 0]$	$[2, 0]$

Stretch: first output $= 2 \times 2 + 0 \times 1 = 4$; second output $= 0 \times 2 + 1 \times 1 = 1$. Rotation preserves length $\sqrt{5}$. Projection loses the second coordinate.

Try the projection with inputs $[2, 1]$ and $[2, -3]$. Both produce $[2, 0]$. Their original second coordinates have been discarded. In contrast, a rotation preserves lengths and can be undone by rotating back.

The experiment uses **column vectors**, with A on the left. For batches stored as rows, a library may use a different placement or transpose. Check the stated convention before copying a formula.

8. Understand independence and lost information

The **span** of some vectors is everything you can form from their linear combinations. Vectors are **linearly independent** if none adds a redundant direction expressible using the others. A **basis** is an independent collection that spans the space you are describing.

The standard basis $[1, 0]$, $[0, 1]$ spans the whole plane. The pair $[1, 0]$, $[2, 0]$ spans only the horizontal line. A matrix's **rank** is the number of independent columns, also equal to the number of independent rows. Identity has rank 2 here; the projection has rank 1. This meaning of rank differs from "number of tensor axes."

The **identity matrix** I leaves every vector unchanged. An **inverse** A^{-1} , when it exists, reverses a square matrix: $A^{-1}A = I$. A square matrix is invertible exactly when it has full rank. Our projection is **singular**, meaning not invertible; no unique inverse can recover its discarded coordinate.

For a 2×2 matrix, the **determinant** is $ad - bc$:

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc$$

Its absolute value is the area scale factor; its sign indicates whether orientation flips. The horizontal stretch has determinant 2, the quarter-turn rotation has determinant 1, and the projection has determinant 0. Zero means a square matrix loses at least one independent direction. A determinant is not defined for a rectangular matrix.

▼ Go deeper: solve equations, least squares, and conditioning

A system $Az = t$ asks for unknown coordinates z . For $p + q = 3$ and $2p - q = 0$, add the equations to get $3p = 3$, hence $p = 1$ and $q = 2$. Substituting back verifies both equations. The coefficient matrix has rows $[1, 1]$ and $[2, -1]$; its determinant is -3 , so the solution is unique.

Dependent equations can have infinitely many solutions or none: $p + q = 3$ and $2p + 2q = 6$ agree but do not fix p and q separately. Replace the final 6 with 7 and they conflict.

Real data often gives more equations than unknown weights, with no exact fit. **Least squares** chooses weights minimizing the sum of squared residuals, the differences between fitted and observed values. It can be viewed as projecting the target onto the span of the design matrix's columns. We revisit fitting with regression.

Do not assume that explicitly computing a matrix inverse is the best way to solve a system. Numerical solvers and least-squares routines avoid unnecessary inversion. An **ill-conditioned** problem is sensitive to small changes in inputs or rounding; invertible does not mean numerically easy. See NumPy's least-squares interface.

▼ Go deeper: eigenvectors, singular values, and compression

A nonzero **eigenvector** of a square matrix satisfies $Av = \lambda v$: the output stays on the same line, scaled by its **eigenvalue** λ . A negative λ reverses direction; zero collapses that vector. For the horizontal stretch, $[1, 0]$ has eigenvalue 2 and $[0, 1]$ has eigenvalue 1. Not every real matrix has a complete basis of real eigenvectors.

Singular value decomposition (SVD) factors a real matrix as $A = U\Sigma V^T$. The orthogonal factors U and V preserve Euclidean geometry; Σ contains nonnegative singular values that describe stretching along selected directions. This works for rectangular matrices too. Keeping only the largest singular values gives a lower-rank approximation: useful for compression, with information loss.

For the stretch matrix $\text{diag}(2, 1)$, the singular values are 2 and 1. Keeping only 2 gives $\text{diag}(2, 0)$, which discards the vertical component. For mean-centered data, **PCA** chooses directions of greatest variance; SVD provides a way to compute them. We will study variance and PCA in their own chapters. See [NumPy's SVD reference](#).

9. Put the calculation into Python

You can implement the essential arithmetic without a numerical library:

```
def dot(a, b):
    if len(a) != len(b):
        raise ValueError("Vector lengths must match.")
    return sum(left * right for left, right in zip(a, b))

features = [[1, 0], [2, 1], [3, 2]]
weights = [2, 0.5]
bias = 1
predictions = [dot(row, weights) + bias for row in features]
print(predictions) # [3.0, 5.5, 8.0]
```

`zip` pairs corresponding entries. Its normal behavior stops at the shorter input, which is why the length check matters. The comprehension calculates one prediction per row; no parameter update occurs.

Extract the workbook, open a terminal in that folder, and run:

```
py linear_algebra.py
```

Use Python 3.10 or newer, substituting `python` or `python3` if that is your working command. No packages are needed. The script prints the batch predictions, length 5 for $[3, 4]$, cosine 1 for the parallel pair, rotated $[2, 1] \rightarrow [-1, 2]$, and projected $[2, 1] \rightarrow [2, 0]$. It writes no files.

Its functions make shape checks and undefined cases explicit. The workbook uses loops so you can inspect the arithmetic. **Vectorization** expresses operations over arrays and lets a library use optimized implementations; it does not mean the mathematical operation stopped doing multiplication and addition.

▼ Connect the shapes to a neural-network layer

Suppose X contains m examples and d input features. A weight matrix W of shape $d \times h$ maps each row to h outputs, giving XW of shape $m \times h$. Add a bias vector with h entries across the rows. An activation function then changes the outputs, usually nonlinearly; we will explain those functions in the neural-network chapter.

For $m = 3$, $d = 2$, $h = 4$, the shape calculation is $(3 \times 2)(2 \times 4) \rightarrow (3 \times 4)$. Four output features do not create four new independent input directions through a linear map: rank cannot exceed the input rank. This is also why merely stacking affine layers without a nonlinear activation can be combined into one affine map.

10. Practice: calculate, check, explain

Exercise 1: calculate a weighted prediction

Use $x = [4, 2]$, $w = [2, 0.5]$, and $b = 1$. What is the prediction? What would happen if you swapped the input entries but kept the weights unchanged?

▼ Reveal the calculation

The intended prediction is $2 \times 4 + 0.5 \times 2 + 1 = 10$ seconds. Swapping gives $2 \times 2 + 0.5 \times 4 + 1 = 7$. Both shapes are valid, but the swapped feature meanings violate the contract.

Exercise 2: predict an output shape

X holds five examples with three features each. W maps those three features to two outputs. What are the shapes of X , W , and XW ? Would a 4×2 W work instead?

▼ Reveal the shape check

X is 5×3 , W is 3×2 , and XW is 5×2 . A 4×2 W fails because the inner dimensions 3 and 4 do not match. Each output combines the same three input features using its own weights.

Exercise 3: identify information loss

The projection matrix has rows $[1, 0]$ and $[0, 0]$. Compare its outputs for $[2, 1]$ and $[2, -3]$. Can you reconstruct which input was used from the output alone?

▼ Reveal the explanation

Both become $[2, 0]$. You cannot distinguish the inputs from that output. The transformation has rank 1, determinant 0, and no inverse. This is a mathematical loss of information, not a rounding error.

11. Check your understanding

Answer all five questions, then read the explanations.

01 What is the dot product of $[3, 2]$ and $[2, 0.5]$?

- A. $[6, 1]$
- B. 8
- C. 7

Answer: C. Multiply corresponding entries and add: $3 \times 2 + 2 \times 0.5 = 7$. The vector $[6, 1]$ is the elementwise product. A separate bias of 1 would make the prediction 8.

02 X has shape 5×3 and W has shape 3×2 . What is the shape of XW?

- A. 5×2
- B. 3×3
- C. 2×5

Answer: A. The inner dimensions match. Five example rows are mapped to two output columns, giving 5×2 .

03 Two nonzero vectors have cosine similarity 1. Must they have the same length?

- A. Yes. They must be identical.
- B. No. They have the same direction but may differ in magnitude.
- C. No. They must be perpendicular.

Answer: B. Cosine divides out the lengths. For example, $[3, 4]$ and $[6, 8]$ have lengths 5 and 10 but cosine similarity 1.

04 A projection turns both $[2, 1]$ and $[2, -3]$ into $[2, 0]$. Can its output uniquely identify the input?

- A. Yes, if we use more decimal places.
- B. Yes, by taking the transpose.
- C. No. It discarded information and has no inverse on the whole plane.

Answer: C. The second coordinate was lost. No amount of precision or transposition restores which of those inputs was used.

05 Can the zero vector be normalized to unit length by dividing by its norm?

- A. Yes; its norm is 1.
- B. No; its norm is 0, so that division is undefined.
- C. Yes; choose any direction without changing the vector.

Answer: B. The zero vector has zero length and no direction. Dividing it by its norm cannot produce a unit vector; the operation is undefined.

12. Keep the meaning beside the mathematics

Before trusting a numerical result, name the axes, check the shape, identify the operation, and explain the units. Then ask what it establishes: a calculation can be mathematically correct while its features, model assumptions, or real-world predictions are wrong.

▼ Quick-reference glossary

Term	Recall cue
Scalar / vector / matrix	One number / ordered coordinates / rectangular array
Shape / transpose	Sizes of axes / exchange row and column positions
Dot product	Pairwise products added to one scalar
Norm / distance	Vector length / length of a difference
Cosine similarity	Compare direction for nonzero vectors
Linear / affine	Preserve addition and scaling / also allow an offset
Span / basis	Reachable combinations / independent spanning vectors
Rank	Number of independent rows or columns
Inverse / singular	Undo a square transformation / cannot invert it
Eigenvector / singular value	A preserved line / a stretch factor in SVD

All examples and diagrams are original. Runtime data is synthetic, and the geometric vectors are illustrations rather than actual embeddings. References beside the optional topics support further study.

Next: Chapter 7: Probability and statistics for AI introduces uncertainty, distributions, and the evidence behind a result.